MIND MATRIX

NAME: MOHAMMED SAHIL R

EMAIL ID: mohammedsahilr39@gmail.com

SESSION: 08

REPORT ON: TASK REPORT

TASK NO: 01

TITLE:

AUTOMATED TESTING DEEP DIVE & MAPPING

1. Key Definitions

- **Unit Test**
  A unit test verifies the functionality of a small piece of logic such as a function without involving the UI.
  Example: Testing a function that calculates a tip amount.
- **Instrumented UI Test**
  An instrumented UI test verifies user interface behavior when a user interacts with UI elements on a device or emulator.
  Example: Checking whether clicking a button changes displayed text.
- **ComposeTestRule**
  ComposeTestRule is used to test composables by launching them in a testing environment and performing UI actions.
  Example:

  ```
  @get:Rule
  val composeTestRule = createComposeRule()
  ```

- **Semantics Tree**
  The semantics tree represents UI components in a structured hierarchy used for accessibility and UI testing.

  Example: Finding elements using onNodeWithText().

2. Selected UI Feature

Feature: **Next Artwork Button in Art Space App**

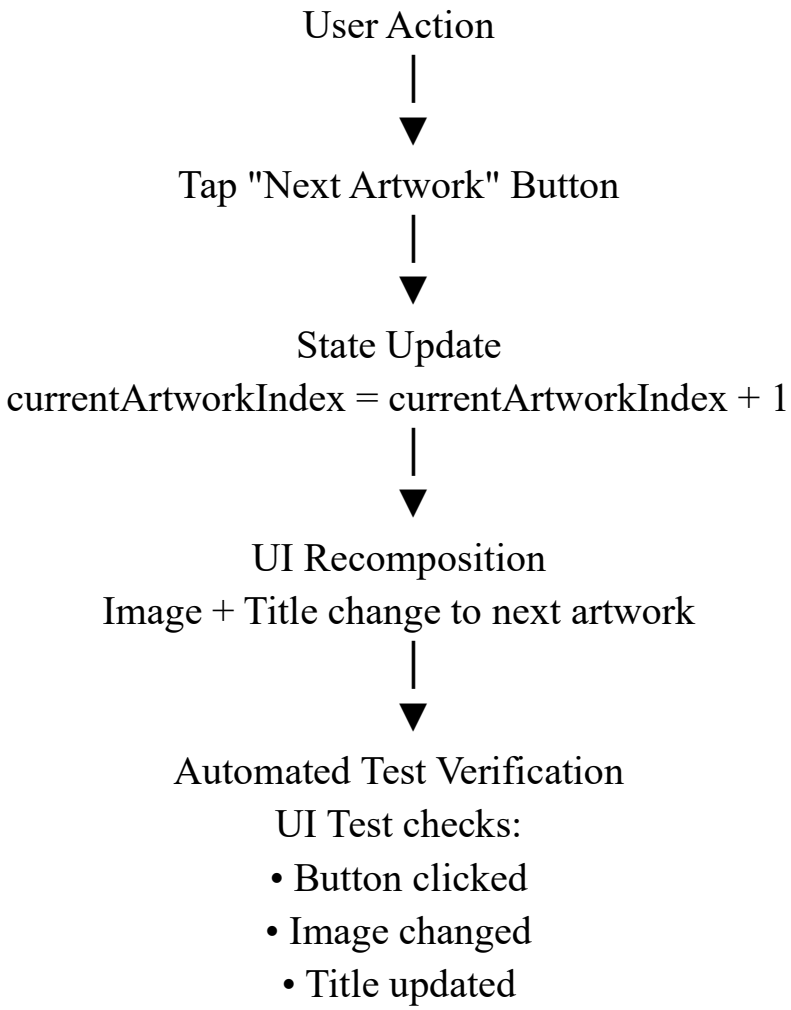
UI Components involved:

- Button
- Image
- Text

Expected Behavior:
When the user presses the **Next** button, the artwork image and title should update to the next artwork.

3. UI Interaction & Testing Flow Diagram

Figure 1: UI Interaction and Test Verification Flow



4. Test Strategy Table

Test Type	What to Test	Setup Needed	Assertion
Unit Test	Artwork index update	Function with index variable	Index increases
UI Test	Button click	ComposeTestRule	Image changes
UI Test	Text display	Launch screen	Correct artwork title

5. Reflection

Writing tests early improves code quality because it ensures features work correctly before deployment. Without UI verification, state changes might occur but the user interface may not update correctly. One challenge in Compose testing is identifying elements in the semantics tree and handling asynchronous UI updates.